

Testr 2.0

Generators (Java)

 [Voltar](#)**Deadline:** Dia 14 de fevereiro de 2026 às 00:00 hora**Linguagem de Programação:****Descrição:**

Generators (geradores) são estruturas de dados utilizadas para produzir sequências de dados por demanda, isto é, sem computar a priori todos os valores. O uso de generatos está relacionado à técnica de lazy computing (computação preguiçosa) que tem como objetivo minimizar o consumo de memória (mantendo um ou poucos itens da sequência na memória de cada vez) e postergar a computação até quando ela é realmente necessária. Para estudar esta técnica, vamos implementar alguns geradores.

Crie uma interface Generator contendo os métodos listados abaixo. Use template/generics para definir o tipo de retorno do método next.

- void reset(): inicia o processo de geração.
- bool is_over(): retorna True se não existem mais itens a serem gerados e False, caso contrário.
- T next(): retorna o próximo item da sequência e atualiza o estado do gerador.

Em seguida, escreva as seguintes classes implementado a interface:

Range (herda de Generator): usado para gerar números sequencialmente.

- No construtor, receba o valor inicial, o valor final e o passo de atualização da sequência. Crie um atributo privado valor_atual que irá armazenar o próximo valor a ser retornado pelo gerador.
- reset(): inicializa o valor_atual como o valor inicial da geração.
- is_over(): compara o valor_atual com o valor final para verificar a

geração deve terminar. Se o passo for positivo e o valor_atual for maior ou igual que o valor final, retorna verdadeiro. De forma similar, se o passo for negativo e o valor_atual for menor ou igual que o valor final, retorna verdadeiro. Se nenhuma das duas condições for satisfeita, retorna Falso.

- next(): retorna o valor_atual e atualiza o valor somando o passo.

Fibonacci (herda de Generator): usado para gerar números de Fibonacci (1, 1, 2, 3, 5, 8, 13, 21, ...) sequencialmente, isto é, um de cada vez.

- No construtor, recebe o número de itens da sequência a serem gerados. Crie atributos privados para armazenar a quantidade de itens produzidos e para guardar os dois últimos valores produzidos.
- reset(): inicializa a quantidade de itens produzidos como 0 e os dois últimos valores produzidos como 1.
- is_over(): verifica se a quantidade de itens produzidos é igual à quantidade desejada.
- next(): Um item da sequência de fibonacci é dado pela soma dos dois anteriores.

Permutations (herda de Generator >): usado para gerar todas as permutações de uma sequência de números. A cada chamada de next retorna a próxima permutação dos números.

- No construtor, recebe o tamanho da sequência a ser permutada.
- reset(): inicializa um array de números inteiros contendo valores de 0 até o tamanho passado no construtor (intervalo aberto, isto é, se foi passado 10, o array deve conter valores de 0 a 9).
- is_over(): termina quando os valores do array estiverem em ordem decrescente (no exemplo, [9, 8, ..., 2, 1, 0]).
- next(): Retorne a sequência atual e atualize o array para ser a próxima permutação (veja a seguir).

Algoritmo para Próxima Permutação

1. Encontrar o Pivô: Percorra o array da direita para a esquerda e

encontre o primeiro índice i tal que $\text{array}[i] < \text{array}[i + 1]$. Esse é o ponto onde a ordem decrescente, vinda do final do array, é quebrada. Os elementos de $\text{array}[i + 1]$ até o fim estão em ordem não crescente. Se não existir tal índice i , significa que todo o array está em ordem decrescente, ou seja, ele já é a última permutação possível.

2. Encontrar o Elemento para Troca Se um pivô i foi encontrado, percorra novamente o array da direita para a esquerda para encontrar o primeiro índice j tal que $\text{array}[j] > \text{array}[i]$. Esse $\text{array}[j]$ será o menor elemento na parte direita que ainda é maior que $\text{array}[i]$.

3. Trocar e Reverter o Sufixo: Troque os valores nos índices i e j . Inverta a sequência de elementos do índice $i + 1$ até o final do array. Esse passo ordena de forma eficiente a parte direita em ordem crescente (já que ela estava em ordem decrescente antes da troca), garantindo que a permutação resultante seja a próxima maior possível.

Exemplo:

Considere que o array atual é $[1, 3, 5, 4, 2]$:

1. Encontrar o Pivô: Percorrendo da direita, o primeiro número menor que seu vizinho da direita é 3 (no índice $i = 1$), pois $3 < 5$. O sufixo é $[5, 4, 2]$.
2. Encontrar o Elemento de Troca: O menor elemento do sufixo $[5, 4, 2]$ que é maior que 3 é 4 (no índice $j = 3$).
3. Trocar: Troque $\text{array}[1] = 3$ com $\text{array}[3] = 4$. O array vira $[1, 4, 5, 3, 2]$.
4. Reverter o Sufixo: Inverta os elementos do índice 2 até o final: $[5, 3, 2]$ vira $[2, 3, 5]$.

Resultado final: $[1, 4, 2, 3, 5]$.

Repetindo o processo continuamente, serão produzidas todas as permutações de valores.

Na main, cria uma variável do tipo Generator `Generator gen = null;``. O símbolo de ``?` é chamado de ``wildcard`` (curinga) e é usado quando não sabemos o tipo que o template vai assumir no momento de declaração da variável.

Em seguida, leia o nome de um gerador e seus argumentos como indicado abaixo.

- Range
- Fibonacci
- Permutations

Crie um objeto do tipo indicado e atribua à variável do tipo Generator. Note como não é possível saber em tempo de compilação qual será o tipo do objeto que será atribuído à variável do tipo Generator.

Em seguida, usando os métodos da interface, gere todos os itens da sequência e mostre-os na tela. Como os tipos gerados por next serão diferentes dependendo de qual gerador foi criado, armazena o resultado de next em uma variável do tipo Object e conte com o mecanismo de amarração tardia para garantir que a versão correta de toString() será invocada para mostrar os dados na tela.

Arquivos

Não existem arquivos

Enviar Resposta

Nenhum arquivo selecionado.

Relatório

Arquivo	sdsaf.zip
Data da Submissão	Dia 14 de dezembro de 2025 às 17:49 horas
Status	Success

Relatório	
-----------	--

Reavaliar

Entradas e Saídas

Entrada 1

Fibonacci 10

Saída 1

1
1
2
3
5
8
13
21
34
55

Entrada 2

Fibonacci 3

Saída 2

1
1
2

Entrada 3

Saída 3

Permutations 1

[0]

Entrada 4

Permutations 2

Saída 4

[0, 1]

[1, 0]

Entrada 5

Permutations 3

Saída 5

[0, 1, 2]

[0, 2, 1]

[1, 0, 2]

[1, 2, 0]

[2, 0, 1]

[2, 1, 0]

Entrada 6

Saída 6

Permutations 4

[0, 1, 2, 3]
[0, 1, 3, 2]
[0, 2, 1, 3]
[0, 2, 3, 1]
[0, 3, 1, 2]
[0, 3, 2, 1]
[1, 0, 2, 3]
[1, 0, 3, 2]
[1, 2, 0, 3]
[1, 2, 3, 0]

Entrada 7

Range 0 10 1

Saída 7

0.0
1.0
2.0
3.0
4.0
5.0
6.0
7.0
8.0
9.0

Entrada 8

Range 10 -1 -1

Saída 8

10.0
9.0
8.0
7.0
6.0
5.0
4.0
3.0
2.0
1.0

Entrada 9

Saída 9

Range -10 10 2

-10.0
-8.0
-6.0
-4.0
-2.0
0.0
2.0
4.0
6.0
8.0

Entrada 10

Range -0.5 0.6 0.1

Saída 10

-0.5
-0.4
-0.30000000000000004
-0.20000000000000004
-0.10000000000000003
-2.7755575615628914E-17
0.09999999999999998
0.19999999999999998
0.3
0.4

Entrada 11

Fibonacci 1

Saída 11

1

Entrada 12

Saída 12

Fibonacci 2

1

1